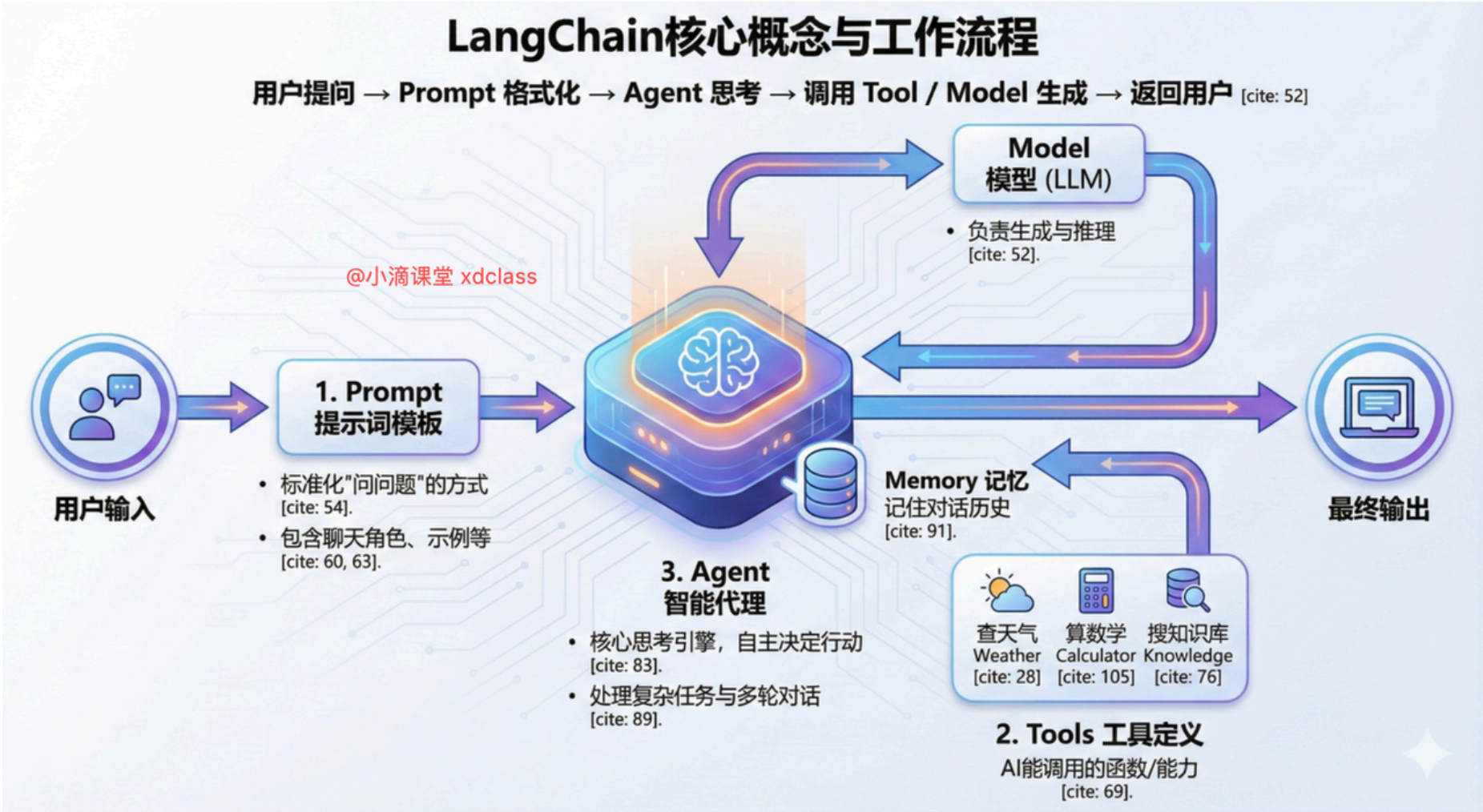


AI开发基础：Langchain框架从入门到实战开发-附代码



LangChain 实战教程：从入门到实战

一份接地气的 LangChain 学习指南，带你从零开始构建 AI 应用

写在前面



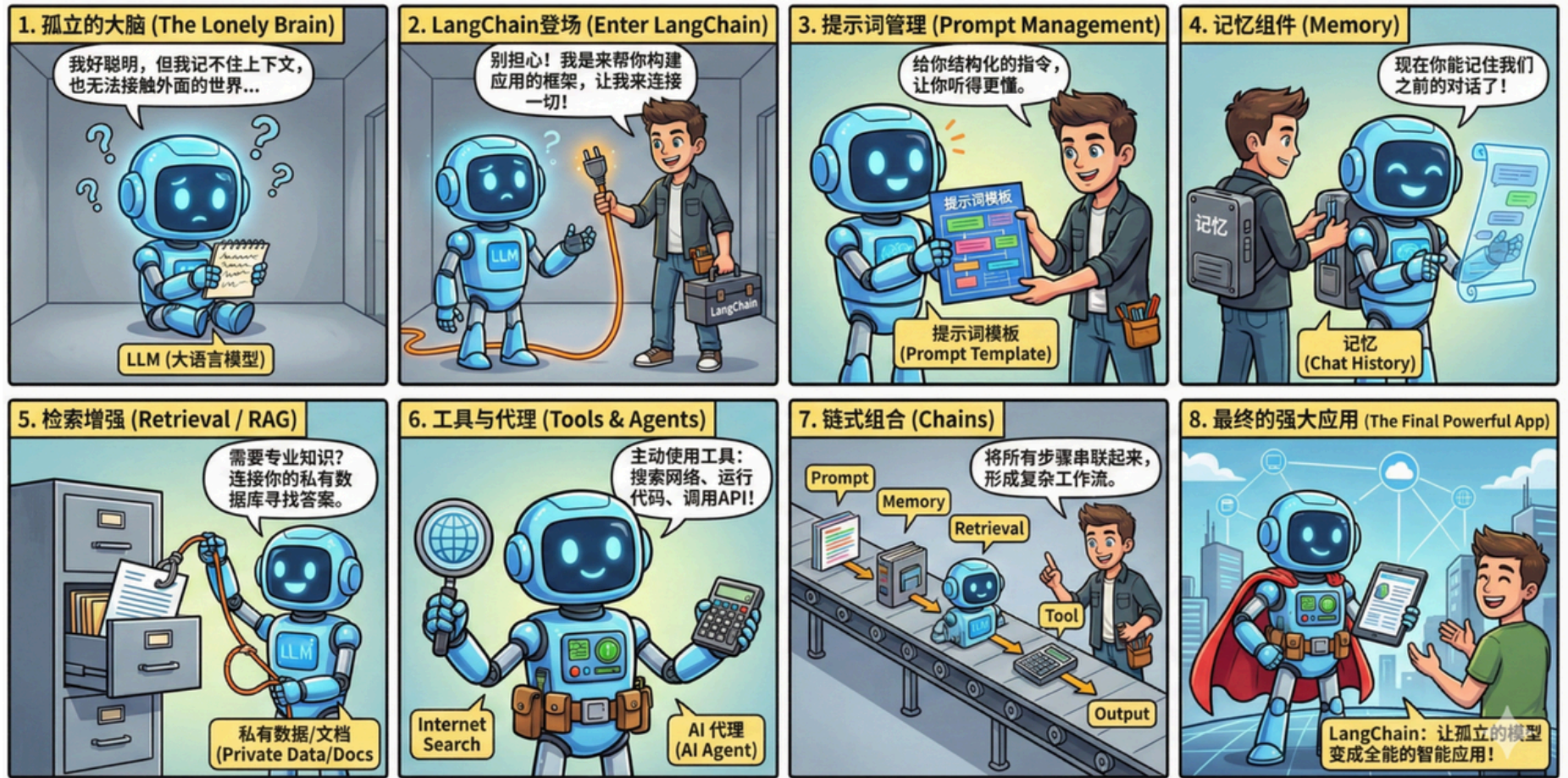
这份教程不会堆砌概念，我会用最直白的话告诉你 LangChain 是什么、怎么用。代码都是能跑的，直接抄就行。

适合学AI应用开发，或者后端转AI开发的想快速上手的同学。

适合谁看：

- 听说过 ChatGPT，想自己做点 AI 应用的开发者
- 被官方文档绕晕的同学
- 想快速上手实战项目的人

什么是 LangChain 框架？（连接大模型与世界的桥梁）



1. LangChain 是什么？

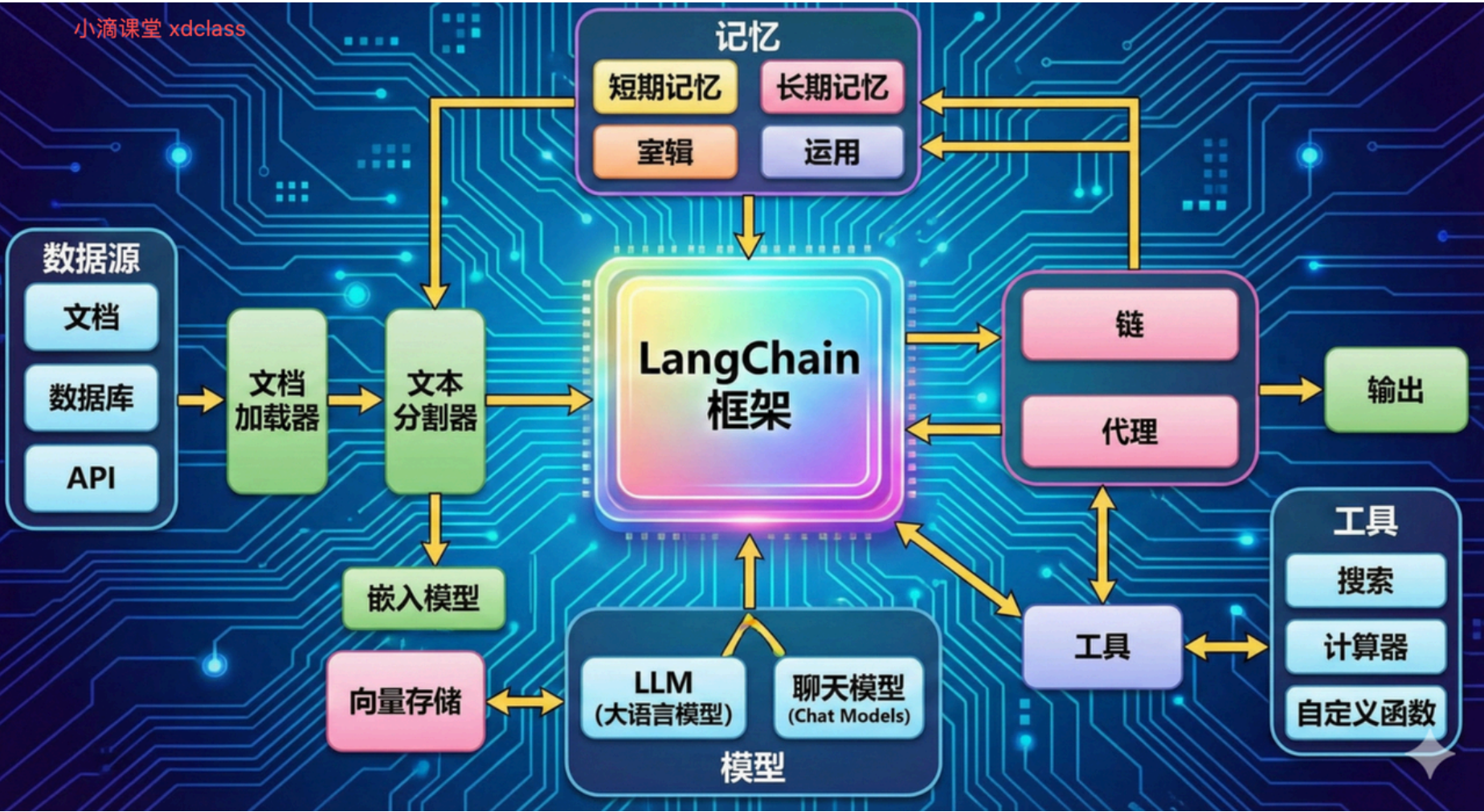
一句话解释

LangChain = 让大模型能"动手干活"的框架

光靠 ChatGPT 只能聊天，但通过 LangChain，你可以让 AI：

- 🔥 查天气、搜网页
- 读文件、写代码
- 调用你公司的内部 API
- 自己决定该做什么

架构图



核心能力

能力	说明	举个例子
模型抽象	一套代码适配多个大模型	今天用 GPT-4，明天换 Claude，代码不用改
工具调用	让 AI 能调用外部功能	AI 可以帮你查股票、发邮件、操作数据库
记忆管理	让 AI 记住对话历史	多轮对话不会"失忆"
Agent	让 AI 自主决策	给它一个目标，它自己规划怎么完成

2. 环境搭建

安装依赖

代码块

```
1  # 创建虚拟环境 (推荐)
2  python -m venv langchain_env
3  source langchain_env/bin/activate # Mac/Linux
4  # langchain_env\Scripts\activate # Windows
5
6  # 安装 LangChain 核心包
7  pip install langchain langchain-core langchain-community
8
9  # 根据你用的模型安装对应包
10 pip install langchain-openai # OpenAI
11 pip install langchain-anthropic # Claude
12 pip install dashscope # 通义千问
13
```

```
14  # 其他常用包
15  pip install python-dotenv          # 环境变量管理
16  pip install langgraph              # Agent 运行时（新版必装）
```

配置 API Key

创建 `.env` 文件：

```
代码块

1  # OpenAI
2  OPENAI_API_KEY=sk-xxx
3
4  # 或者通义千问
5  DASHSCOPE_API_KEY=sk-xxx
6
7  # Claude
8  ANTHROPIC_API_KEY=sk-xxx
```

验证安装

```
代码块

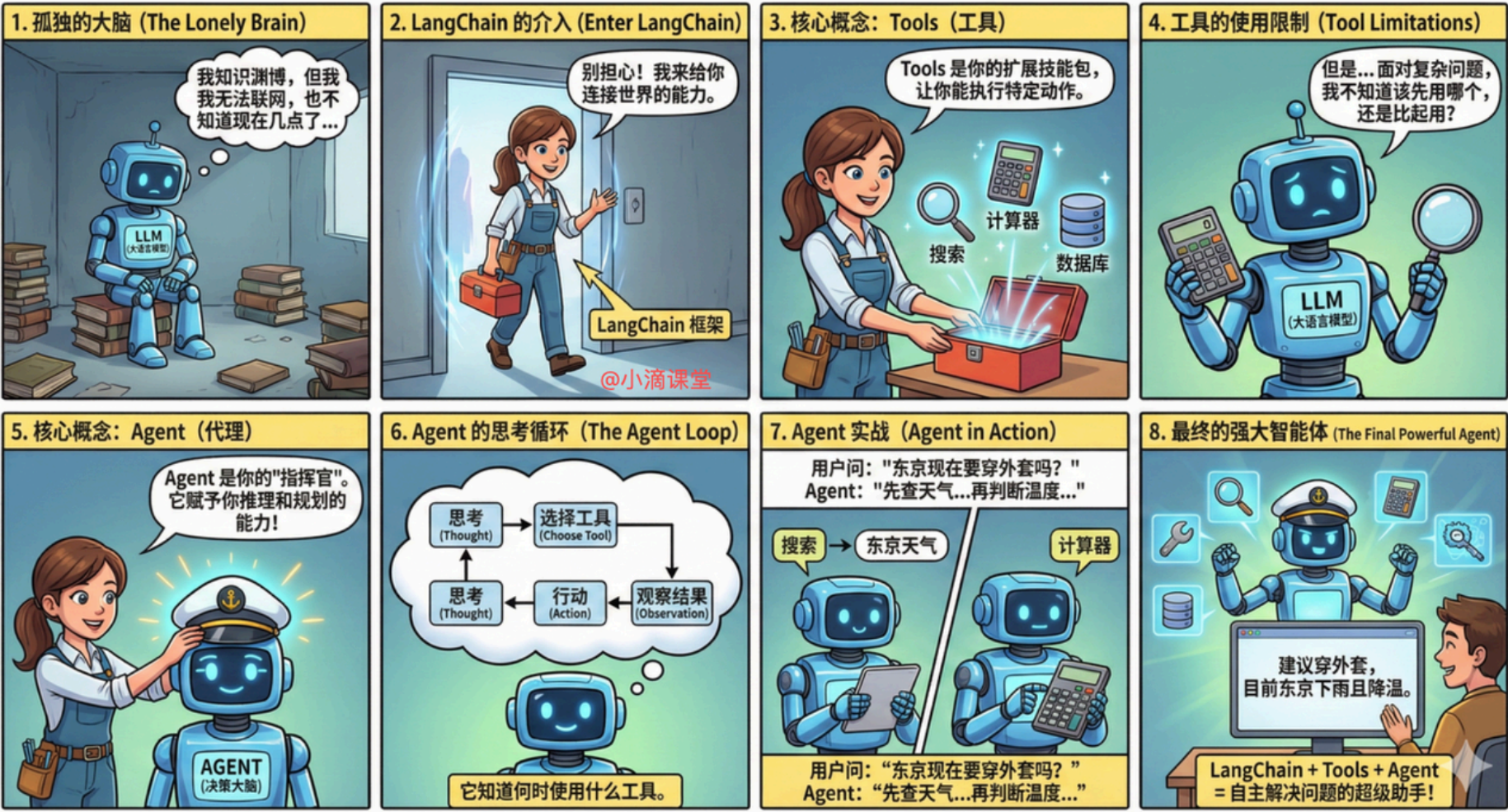
1  from dotenv import load_dotenv
2  import os
3
4  load_dotenv()
5
6  # 通义千问示例
7  from langchain_community.chat_models import ChatTongyi
8
9  llm = ChatTongyi(
10      model="qwen-turbo",
11      dashscope_api_key=os.getenv("DASHSCOPE_API_KEY")
12  )
13
14  response = llm.invoke("你好，请用一句话介绍自己")
15  print(response.content)
```

跑通了就说明环境 OK！

3. 核心概念速览

 LangChain 的核心就四个东西，搞懂了就入门了：

概念地图



四大核心

概念	是什么	类比
Model	大语言模型	大脑
Prompt	提示词模板	问题格式
Tool	外部工具	双手
Agent	智能代理	管家，帮你协调一切

关系是这样的：

代码块

1 用户提问 → Prompt 格式化 → Agent 思考 → 调用 Tool → Model 生成回答 → 返回用户

4. Prompt 提示词模板

🔥 提示词模板就是把"问问题"这件事标准化。不用每次都手写完整的提示词。

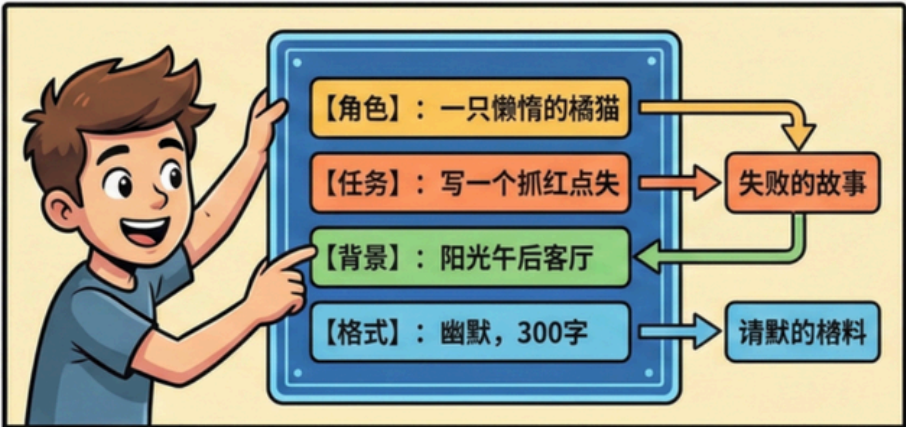
工作流程



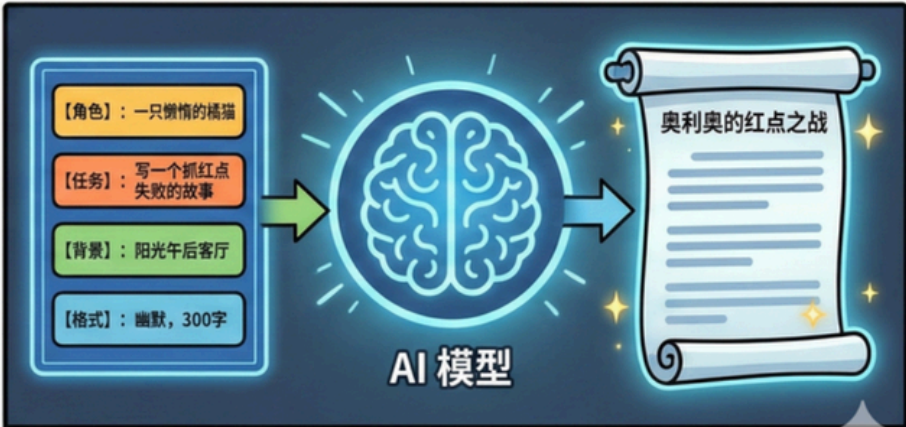
1. 没有框架：提示词混乱，AI 难以理解意图。



2. 核心概念一：结构化模板 (Structured Templates)。提供清晰的插槽，引导思考。



3. 核心概念二：变量替换 (Variable Substitution)。将具体信息填入预设位置。



4. 核心概念三：复用与优化 (Reuse & Optimization)。生成高质量、可复用的 Prompt，提升 AI 输出效果！

4.1 最简单的模板

```
代码块

1  from langchain_core.prompts import PromptTemplate
2
3  # 定义模板，用 {} 占位
4  template = PromptTemplate.from_template(
5      "请用{language}语言介绍一下{topic}，不超过100字。"
6  )
7
8  # 填充变量
9  prompt = template.format(language="中文", topic="人工智能")
10 print(prompt)
11 # 输出： 请用中文语言介绍一下人工智能，不超过100字。
```

4.2 聊天模板（常用）

做聊天应用一般用这个，可以设置 system 和 user 角色：

```
代码块

1  from langchain_core.prompts import ChatPromptTemplate
2
3  # 定义多轮对话模板
4  chat_template = ChatPromptTemplate.from_messages([
5      ("system", "你是一位{role}，擅长用简洁易懂的方式解释复杂概念。"),
6      ("human", "请解释一下：{concept}"),
7  ])
8
9  # 生成消息列表
10 messages = chat_template.format_messages(
11     role="物理学教授",
12     concept="量子纠缠"
13 )
14
```



```
15  # 调用模型
16  response = llm.invoke(messages)
17  print(response.content)
```

4.3 Few-shot 示例模板

通过给几个例子，教 AI 按你想要的格式输出：

```
代码块

1  # 定义示例
2  examples = [
3      {"input": "开心", "output": "我今天非常开心! "},
4      {"input": "难过", "output": "我感到有些难过..."},
5  ]
6
7  # 创建包含示例的模板
8  few_shot_template = ChatPromptTemplate.from_messages([
9      ("system", "你是一个情绪表达助手。"),
10     ("human", "示例：\n情绪：开心\n表达：我今天非常开心! \n\n情绪：难过\n表达：我感到有些难过..."),
11     ("human", "现在请表达这个情绪：{emotion}")
12 ])
13
14 messages = few_shot_template.format_messages(emotion="兴奋")
15 response = llm.invoke(messages)
```

4.4 动态日期模板

有些变量想自动填充（比如当前日期）：

```
代码块

1  from datetime import datetime
2  from langchain_core.prompts import PromptTemplate
3
4  template = PromptTemplate(
5      template="今天是{date}，请告诉我关于{topic}的最新消息。",
6      input_variables=["topic"],
7      partial_variables={
8          "date": datetime.now().strftime("%Y年%m月%d日") # 自动填充
9      }
10 )
11
12 # 只需要传 topic
13 prompt = template.format(topic="AI发展")
```

5. Tools 工具定义

工具就是让 AI 能调用的函数。定义好工具后，AI 会自己决定什么时候用、怎么用。

5.1 用装饰器定义（最简单）

```
代码块

1  from langchain_core.tools import tool
2
3  @tool
4  def get_weather(city: str) -> str:
```

```

5         """
6         获取城市的天气信息
7
8         Args:
9             city: 城市名称, 如"北京"、"上海"
10        """
11        # 模拟天气数据 (实际项目中调用天气 API)
12        weather_data = {
13            "北京": "晴天, 25°C",
14            "上海": "多云, 22°C",
15            "深圳": "小雨, 28°C",
16        }
17        return weather_data.get(city, f"{city}的天气暂时无法获取")
18
19
20    # 测试工具
21    result = get_weather.invoke({"city": "北京"})
22    print(result)  # 输出: 晴天, 25°C
23
24    # 查看工具信息
25    print(f"工具名称: {get_weather.name}")
26    print(f"工具描述: {get_weather.description}")
27    print(f"参数结构: {get_weather.args}")

```

重点: docstring 写清楚! AI 就是靠这个描述来决定什么时候调用这个工具。

5.2 更复杂的工具

代码块

```

1  from typing import Optional
2
3  @tool
4  def search_database(query: str, category: Optional[str] = None) -> str:
5      """
6      在数据库中搜索信息
7
8      Args:
9          query: 搜索关键词
10         category: 可选的分类过滤器, 如"科技"、"健康"、"教育"
11     """
12     database = {
13         "科技": ["人工智能正在改变世界", "5G技术的应用"],
14         "健康": ["健康饮食的重要性", "运动与长寿的关系"],
15         "教育": ["在线学习的趋势", "终身学习的价值"]
16     }
17
18     results = []
19
20     if category and category in database:
21         for item in database[category]:
22             if query.lower() in item.lower():
23                 results.append(f"[{category}] {item}")
24     else:
25         for cat, items in database.items():
26             for item in items:
27                 if query.lower() in item.lower():
28                     results.append(f"[{cat}] {item}")
29
30     if results:
31         return "找到以下结果:\n" + "\n".join(results)

```



```
32         else:
33             return f"没有找到关于 '{query}' 的结果"
```

5.3 用类定义（需要更多控制时）

代码块

```
1  from langchain_core.tools import BaseTool
2  from pydantic import BaseModel, Field
3  from typing import Type
4
5  # 定义输入参数结构
6  class CalculatorInput(BaseModel):
7      expression: str = Field(description="数学表达式，如 '2+2' 或 '10*5'")
8
9  class Calculator(BaseTool):
10     name: str = "calculator"
11     description: str = "执行数学计算"
12     args_schema: Type[BaseModel] = CalculatorInput
13
14     def _run(self, expression: str) -> str:
15         """同步执行"""
16         try:
17             result = eval(expression)
18             return f"计算结果: {expression} = {result}"
19         except Exception as e:
20             return f"计算错误: {str(e)}"
21
22     async def _arun(self, expression: str) -> str:
23         """异步执行（可选实现）"""
24         return self._run(expression)
25
26 # 使用
27 calc = Calculator()
28 result = calc.invoke({"expression": "123 * 456"})
29 print(result) # 计算结果: 123 * 456 = 56088
```

5.4 把工具绑定到模型

定义好工具后，需要"告诉"模型这些工具的存在：

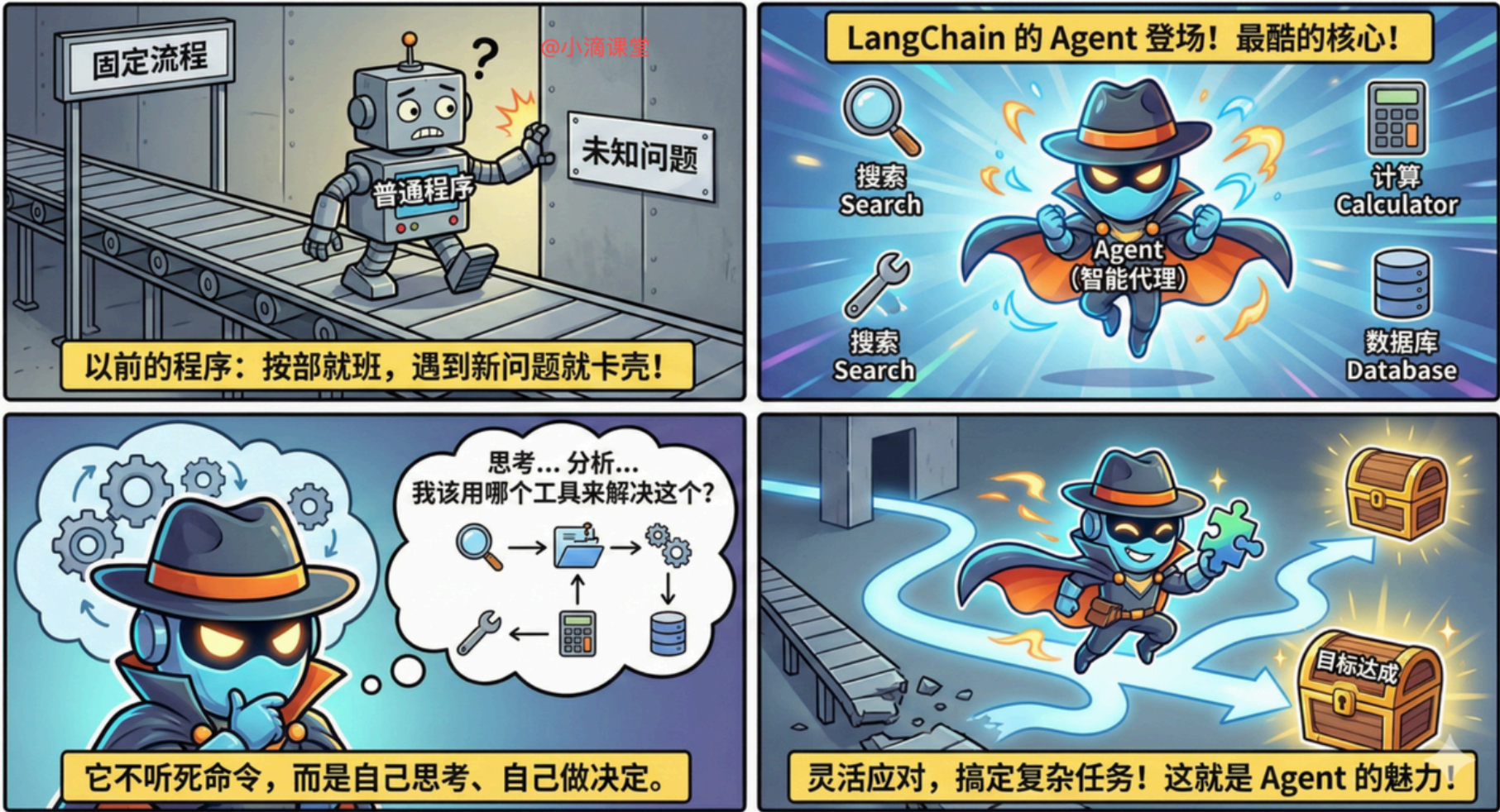
代码块

```
1  # 创建工具列表
2  tools = [get_weather, search_database, Calculator()]
3
4  # 绑定到模型
5  llm_with_tools = llm.bind_tools(tools)
6
7  # 现在模型知道有这些工具可用了
8  response = llm_with_tools.invoke("北京今天天气如何? ")
9
10 # 检查模型是否决定调用工具
11 if response.tool_calls:
12     for tool_call in response.tool_calls:
13         print(f"模型想调用: {tool_call['name']}")
14         print(f"传入参数: {tool_call['args']}")
```

6. Agent 智能代理

🔥 Agent 是 LangChain 最酷的部分。它不是按固定流程执行，而是自己思考、自己决定该干啥。

Agent 思考循环



6.1 LangChain 1.0+ 新版 Agent

新版用 `create_agent`，底层基于 LangGraph，更稳定：

代码块

```
1 from langchain.agents import create_agent
2 from langchain_core.tools import tool
3 from langchain_community.chat_models import ChatTongyi
4 import os
5
6 # 1. 初始化模型
7 llm = ChatTongyi(
8     model="qwen-plus",
9     temperature=0.7,
10    dashscope_api_key=os.environ.get("DASHSCOPE_API_KEY")
11 )
12
13 # 2. 定义工具
14 @tool
15 def get_weather(city: str) -> str:
16     """获取城市天气信息"""
17     weather_db = {
18         "北京": "晴天, 15-25度",
19         "上海": "多云, 18-28度",
20         "深圳": "小雨, 20-30度",
21     }
22     return weather_db.get(city, f"{city}的天气信息暂不可用")
23
24 @tool
25 def calculator(expression: str) -> str:
```



```

26     """执行数学计算"""
27     try:
28         result = eval(expression)
29         return f"计算结果: {expression} = {result}"
30     except:
31         return "计算错误"
32
33 @tool
34 def search_knowledge(query: str) -> str:
35     """搜索知识库"""
36     knowledge = {
37         "LangChain": "LangChain是一个用于开发LLM应用的框架，支持工具、代理、内存管理等功能。",
38         "机器学习": "机器学习是AI的子集，让系统能从数据中自动学习和改进。",
39     }
40     for key, value in knowledge.items():
41         if key in query:
42             return value
43     return f"未找到关于'{query}'的信息"
44
45 # 3. 创建 Agent
46 agent = create_agent(
47     model=llm,
48     tools=[get_weather, calculator, search_knowledge],
49     system_prompt="你是一个专业的中文助手。仔细分析用户问题，选择合适的工具来回答。"
50 )
51
52 # 4. 使用 Agent
53 result = agent.invoke({
54     "messages": [{"role": "user", "content": "北京今天天气怎么样? "}])
55
56
57 # 获取回答
58 final_message = result["messages"][-1]
59 print(f"回答: {final_message.content}")

```

6.2 Agent 处理多个问题

代码块

```

1  test_queries = [
2      "北京今天天气怎么样? ",
3      "给我讲讲什么是机器学习",
4      "计算 123 * 456",
5  ]
6
7  for query in test_queries:
8      print(f"\n用户: {query}")
9
10     result = agent.invoke({
11         "messages": [{"role": "user", "content": query}]
12     })
13
14     final_message = result["messages"][-1]
15     print(f"助手: {final_message.content}")

```

6.3 带记忆的 Agent

让 Agent 记住对话历史：

```

1  from langgraph.checkpoint.memory import MemorySaver
2
3  # 创建记忆存储
4  checkpointer = MemorySaver()
5
6  # 创建带记忆的 Agent
7  agent = create_agent(
8      model=llm,
9      tools=[get_weather, calculator, search_knowledge],
10     system_prompt="你是一个专业助手，能记住之前的对话。",
11     checkpointer=checkpointer # 启用记忆
12 )
13
14 # 会话配置 (同一个 thread_id 共享记忆)
15 config = {"configurable": {"thread_id": "session_001"}}
16
17 # 第一轮对话
18 result1 = agent.invoke(
19     {"messages": [{"role": "user", "content": "我叫小明"}]},
20     config
21 )
22 print(result1["messages"][-1].content)
23
24 # 第二轮对话 (Agent 会记得用户叫小明)
25 result2 = agent.invoke(
26     {"messages": [{"role": "user", "content": "我叫什么名字? "}]},
27     config
28 )
29 print(result2["messages"][-1].content) # 会回答"小明"

```

6.4 流式输出

让回答一个字一个字输出，体验更好：

代码块

```

1  # 流式调用
2  for chunk in agent.stream(
3      {"messages": [{"role": "user", "content": "北京天气如何? "}]},
4      config,
5      stream_mode="values"
6  ):
7      messages = chunk.get("messages")
8      if messages:
9          last_message = messages[-1]
10         if hasattr(last_message, "content") and last_message.content:
11             print(last_message.content, end="", flush=True)
12 print() # 换行

```

7. 完整项目：全能智能助手

现在把前面学的串起来，做一个能干实事的智能助手。

7.1 项目结构

代码块

```

1  smart_assistant/

```



```
2  |— .env                # API Key
3  |— main.py            # 主程序
4  |— tools/             # 工具定义
5  |   |— __init__.py
6  |   |— weather.py
7  |   |— calculator.py
8  |   |— translator.py
9  |   |— knowledge.py
10 |— requirements.txt
```

7.2 工具定义

tools/weather.py

代码块

```
1  from langchain_core.tools import tool
2  from datetime import datetime
3
4  @tool
5  def get_weather(city: str) -> str:
6      """
7      获取指定城市的实时天气信息
8
9      Args:
10         city: 城市名称, 支持北京、上海、广州、深圳、杭州等
11     """
12     weather_database = {
13         "北京": {"condition": "晴天", "temp": "15-25°C", "aqi": "优"},
14         "上海": {"condition": "多云", "temp": "18-28°C", "aqi": "良"},
15         "深圳": {"condition": "小雨", "temp": "22-30°C", "aqi": "优"},
16         "杭州": {"condition": "阴天", "temp": "17-26°C", "aqi": "良"},
17         "广州": {"condition": "晴天", "temp": "20-32°C", "aqi": "良"},
18     }
19
20     if city not in weather_database:
21         return f"暂无{city}的天气数据, 支持城市: 北京、上海、深圳、杭州、广州"
22
23     data = weather_database[city]
24     return f"""
25 {city} 天气预报
26
27 天气: {data['condition']}
28 温度: {data['temp']}
29 空气质量: {data['aqi']}
30 更新时间: {datetime.now().strftime("%H:%M")}
31     """.strip()
```

tools/calculator.py

代码块

```
1  from langchain_core.tools import tool
2
3  @tool
4  def calculator(expression: str) -> str:
5      """
6      执行数学计算
7
8      Args:
```

```

9         expression: 数学表达式, 如 "2+2"、"(10*5)+20"、"2**8"
10         """
11     try:
12         # 安全检查
13         allowed_chars = set('0123456789+-*/(). ')
14         if not all(c in allowed_chars for c in expression.replace('**', '')):
15             return "表达式包含非法字符"
16
17         result = eval(expression, {"__builtins__": {}}, {})
18
19         if isinstance(result, float) and result.is_integer():
20             result = int(result)
21
22         return f"计算结果: {expression} = {result}"
23     except ZeroDivisionError:
24         return "错误: 除数不能为零"
25     except Exception as e:
26         return f"计算错误: {str(e)}"

```

tools/translator.py

代码块

```

1  from langchain_core.tools import tool
2
3  @tool
4  def translate(text: str, target_language: str = "英文") -> str:
5      """
6      将中文文本翻译成其他语言（模拟）
7
8      Args:
9          text: 要翻译的中文文本
10         target_language: 目标语言, 支持"英文"、"日文"、"韩文"
11     """
12     translations = {
13         "你好": {"英文": "Hello", "日文": "こんにちは", "韩文": "안녕하세요"},
14         "谢谢": {"英文": "Thank you", "日文": "ありがとう", "韩文": "감사합니다"},
15         "再见": {"英文": "Goodbye", "日文": "さようなら", "韩文": "안녕히 가세요"},
16     }
17
18     if text in translations and target_language in translations[text]:
19         result = translations[text][target_language]
20         return f"翻译结果: '{text}' -> [{target_language}] {result}"
21
22     return f"暂不支持 '{text}' 的{target_language}翻译"

```

7.3 主程序

main.py

代码块

```

1  import os
2  from dotenv import load_dotenv
3  from langchain.agents import create_agent
4  from langchain_community.chat_models import ChatTongyi
5  from langgraph.checkpoint.memory import MemorySaver
6
7  # 导入工具
8  from tools.weather import get_weather

```

```
9  from tools.calculator import calculator
10 from tools.translator import translate
11
12 # 加载环境变量
13 load_dotenv()
14
15 # 系统提示词
16 SYSTEM_PROMPT = """
17 你是小智，一个全能智能助手。
18
19 你可以帮用户：
20 • 查询天气：查询中国主要城市的天气
21 • 数学计算：进行各种数学运算
22 • 文本翻译：将中文翻译成其他语言
23
24 工作原则：
25 1. 先理解用户意图，选择合适的工具
26 2. 如果不确定，可以询问用户
27 3. 回答简洁明了，有帮助
28 """
29
30 def create_assistant():
31     """创建智能助手"""
32     # 初始化模型
33     llm = ChatTongyi(
34         model="qwen-plus",
35         temperature=0.7,
36         dashscope_api_key=os.environ.get("DASHSCOPE_API_KEY")
37     )
38
39     # 创建记忆
40     checkpointer = MemorySaver()
41
42     # 创建 Agent
43     agent = create_agent(
44         model=llm,
45         tools=[get_weather, calculator, translate],
46         system_prompt=SYSTEM_PROMPT,
47         checkpointer=checkerpointer
48     )
49
50     return agent
51
52 def main():
53     """主函数"""
54     print("=" * 50)
55     print("      小智 - 全能智能助手 v1.0")
56     print("=" * 50)
57     print("输入 'quit' 退出\n")
58
59     agent = create_assistant()
60     config = {"configurable": {"thread_id": "main"}}
61
62     while True:
63         user_input = input("你: ").strip()
64
65         if user_input.lower() in ['quit', '退出', 'exit']:
66             print("再见! ")
67             break
68
69         if not user_input:
70             continue
```



```
71
72     try:
73         result = agent.invoke(
74             {"messages": [{"role": "user", "content": user_input}]},
75             config
76         )
77
78         response = result["messages"][-1].content
79         print(f"小智: {response}\n")
80
81     except Exception as e:
82         print(f"出错了: {str(e)}\n")
83
84 if __name__ == "__main__":
85     main()
```

7.4 运行效果

代码块

```
1  =====
2      小智 - 全能智能助手 v1.0
3  =====
4  输入 'quit' 退出
5
6  你: 北京天气怎么样
7  小智:
8  北京 天气预报
9  _____
10  天气: 晴天
11  温度: 15-25°C
12  空气质量: 优
13  更新时间: 14:30
14
15  你: 帮我算一下 125 * 88
16  小智: 计算结果: 125 * 88 = 11000
17
18  你: 把"你好"翻译成日文
19  小智: 翻译结果: '你好' → [日文] こんにちは
20
21  你: quit
22  再见!
```

8. 进阶技巧与最佳实践

8.1 工具描述要写好

AI 完全靠工具的 description 来决定什么时候调用。写得不清楚，它就不会用。

坏例子：

代码块

```
1  @tool
2  def func(x: str) -> str:
3      """处理数据""" # 太模糊了!
4      pass
```

好例子：

代码块

```
1  @tool
2  def get_stock_price(symbol: str) -> str:
3      """
4      获取股票实时价格
5
6      当用户询问某只股票的价格、涨跌情况时使用此工具。
7
8      Args:
9          symbol: 股票代码，如 "AAPL"（苹果）、"GOOGL"（谷歌）
10
11     Returns:
12         包含当前价格、涨跌幅的字符串
13     """
14     pass
```

8.2 错误处理要完善

工具可能会失败（网络错误、参数错误等），一定要有兜底：

代码块

```
1  @tool
2  def fetch_data(url: str) -> str:
3      """从URL获取数据"""
4      try:
5          response = requests.get(url, timeout=10)
6          response.raise_for_status()
7          return response.text
8      except requests.Timeout:
9          return "请求超时，请稍后重试"
10     except requests.RequestException as e:
11         return f"请求失败: {str(e)}"
12     except Exception as e:
13         return f"未知错误: {str(e)}"
```

8.3 调整温度参数

- `temperature=0`：输出最确定，适合事实查询
- `temperature=0.7`：有一定创意，适合对话
- `temperature=1.0`：输出多样，适合创作

8.4 使用 LangSmith 调试

遇到奇怪问题时，用 LangSmith 看看 Agent 到底在想什么：

代码块

```
1  import os
2
3  # 在代码最前面添加
4  os.environ["LANGCHAIN_TRACING_V2"] = "true"
5  os.environ["LANGCHAIN_API_KEY"] = "你的LangSmith API Key"
```

然后去 smith.langchain.com 就能看到每一步的详细 trace。

8.5 控制迭代次数

防止 Agent 陷入死循环：

代码块

```
1  agent = create_agent(  
2      model=llm,  
3      tools=tools,  
4      # 其他选项  
5  )  
6  
7  # 调用时限制最大步数  
8  result = agent.invoke(  
9      {"messages": [...]},  
10     config,  
11     recursion_limit=10 # 最多10步  
12 )
```

全文评论



彭绍湧 7月13日 20:54

@用户 669719 这个是基础文档 现在有详细的专题课程了 可以找顾问老师获取下